

Sinsera Garage Pi Bridge — Raspberry Pi 5 + OBDLink MX+ + Supabase live OBD-II feed

[0 stars](#) [0 forks](#) [0 watching](#) [Branches](#) [Activity](#)  
[Tags](#)[Public repository](#)[m...](#) [1 Branch](#) [0 Tags](#) [Go to file](#) [T](#) [Go to file](#) [Add file](#) [Code](#) [...](#) **PetaStockdale** feat: Garage Pi Bridge — Raspberry Pi 5 → OBDLink MX+ → Supabase [441f4fe · 4 days ago](#) 

|                                                                                                             |                                              |            |
|-------------------------------------------------------------------------------------------------------------|----------------------------------------------|------------|
|  .env.example              | feat: Garage Pi Bridge — Raspberry Pi 5 →... | 4 days ago |
|  .gitignore                | feat: Garage Pi Bridge — Raspberry Pi 5 →... | 4 days ago |
|  README.md                 | feat: Garage Pi Bridge — Raspberry Pi 5 →... | 4 days ago |
|  bridge.py                 | feat: Garage Pi Bridge — Raspberry Pi 5 →... | 4 days ago |
|  garage-obd-bridge.service | feat: Garage Pi Bridge — Raspberry Pi 5 →... | 4 days ago |
|  install.sh                | feat: Garage Pi Bridge — Raspberry Pi 5 →... | 4 days ago |
|  obd_pids.py               | feat: Garage Pi Bridge — Raspberry Pi 5 →... | 4 days ago |
|  requirements.txt        | feat: Garage Pi Bridge — Raspberry Pi 5 →... | 4 days ago |

[README](#) 

## Garage Pi Bridge

A Raspberry Pi 5 sitting in the car, paired to an **OBDLink MX+** over Bluetooth Low Energy, reading OBD-II PIDs from the ECU and streaming them into Supabase. The Sinsera Garage web app ( [sinsera.co/garage/:carId/obd](https://sinsera.co/garage/:carId/obd) ) subscribes to those rows via Supabase Realtime and renders gauges + DTCs live.

This is the **Pi-bridge** alternative to building a native iOS app — see [Sinsera Core/docs/IOS\\_OBD\\_BRIDGE.md](#) for why a pure web app can't talk to the MX+ directly on iOS.

## Hardware

- **Raspberry Pi 5** (any RAM variant; 4 GB is plenty)
- microSD card, 16 GB+
- 5V/5A USB-C PSU (Pi 5 needs the official one for stable BLE) — or a car-USB-PD adapter
- **OBDLink MX+** plugged into the OBD-II port (under the dash, driver's side)
- Optional but recommended: a USB-C female → 12V cigarette-lighter adapter so the Pi powers off the car battery

## Software stack

- **DietPi** (Debian Trixie ARM64) — minimal Pi distro. Image already at [/Users/petastockdale/Dev-Sinsera/DietPi\\_RPi5-ARMv8-Trixie.img](#)
- **Python 3.11+** with `bleak` (BLE) + `aiohttp` (Supabase REST) + `python-dotenv`

- **bluetoothctl** / **bluez** — already in DietPi
- **systemd** — runs the bridge as a daemon on boot

## End-to-end setup (≈ 20 min)

### 1. Flash the SD card

On your Mac, with the SD card plugged in:

```
# Find the SD card device
diskutil list
# Unmount but don't eject
diskutil unmountDisk /dev/diskN    # whichever N your card is

# Flash. Note: this overwrites the entire card.
sudo dd if=/Users/petastockdale/Dev-Sinsera/DietPi_RPi5-ARMv8-Trixie.img \
of=/dev/rdiskN bs=4m status=progress
```



Or use **Raspberry Pi Imager** GUI — easier; pick "Use custom" → that .img.

### 2. Pre-configure DietPi (before first boot)

DietPi reads `dietpi.txt` from the boot partition on first boot for unattended setup. Mount the boot partition (it auto-mounts as `bootfs` after flashing) and edit:

```
# Boot partition is /Volumes/bootfs on macOS after re-inserting the card
nano /Volumes/bootfs/dietpi.txt
```



Set at minimum:

```
AUTO_SETUP_LOCALE=en_AU.UTF-8
AUTO_SETUP_KEYBOARD_LAYOUT=au
AUTO_SETUP_TIMEZONE=Australia/Brisbane
AUTO_SETUP_NET_WIFI_SSID=<your wifi name>
AUTO_SETUP_NET_WIFI_KEY=<your wifi password>
AUTO_SETUP_HEADLESS=1
AUTO_SETUP_ACCEPT_LICENSE=1
SURVEY_OPTED_IN=0
AUTO_SETUP_AUTOMATED=1
AUTO_SETUP_GLOBAL_PASSWORD=<root password — change this>
```



(For a car-mounted Pi, hotspot to your phone's wifi — set the SSID/key to your phone's hotspot.)

### 3. First boot + SSH in

Insert the SD card in the Pi, power on. First boot takes 5–10 min while DietPi runs its installer.

Once it's on the network, SSH in:

```
ssh root@dietpi.local
# or use the IP from your router's DHCP list
```



### 4. Install the bridge

```
# Pull the bridge code
cd /opt
git clone https://github.com/devsinsera/garage-pi-bridge.git
cd garage-pi-bridge

# Run the installer (installs deps + creates .env + sets up systemd)
sudo bash install.sh
```



## 5. Configure environment

Fill in `/opt/garage-pi-bridge/.env` :

```
SUPABASE_URL=https://<your-project>.supabase.co
SUPABASE_ANON_KEY=eyJ...
SUPABASE_EMAIL=peta.stockdale@outlook.com
SUPABASE_PASSWORD=<your password>
CAR_ID=<GR86 row UUID from garage_cars table>
OBDLINK_NAME=OBDLink MX+      # default; override if your MX+ has a custom name
SAMPLE_HZ=5                   # PIDs read per second
DTC_POLL_SECS=10              # how often to query DTCs
```

Get your `CAR_ID` from the Supabase SQL editor:

```
select id, year, make, model, nickname
from garage_cars
where user_id = auth.uid();
```

## 6. Pair the OBDLink MX+ once

The Pi has to know about the MX+ before the bridge can autoconnect. With the MX+ powered (car ignition ON or accessory mode), run:

```
sudo bluetoothctl
# inside the bluetoothctl shell:
power on
agent on
scan on
# wait ~10s - OBDLink MX+ should appear with its MAC. Copy the MAC.
pair <MAC>
trust <MAC>
exit
```

Paste the MAC into `.env` as `OBDLINK_MAC=` .

## 7. Start the service

```
sudo systemctl restart garage-obd-bridge
sudo systemctl status garage-obd-bridge
journalctl -u garage-obd-bridge -f      # tail logs
```

You should see:

```
[bridge] connecting to OBDLink MX+ ...
[bridge] ELM327 ready, protocol auto-detected (ISO 15765-4 CAN)
[bridge] session a3f...  car 7c1...
[bridge] 5 Hz sampling started - RPM 824, speed 0, coolant 86°C
```

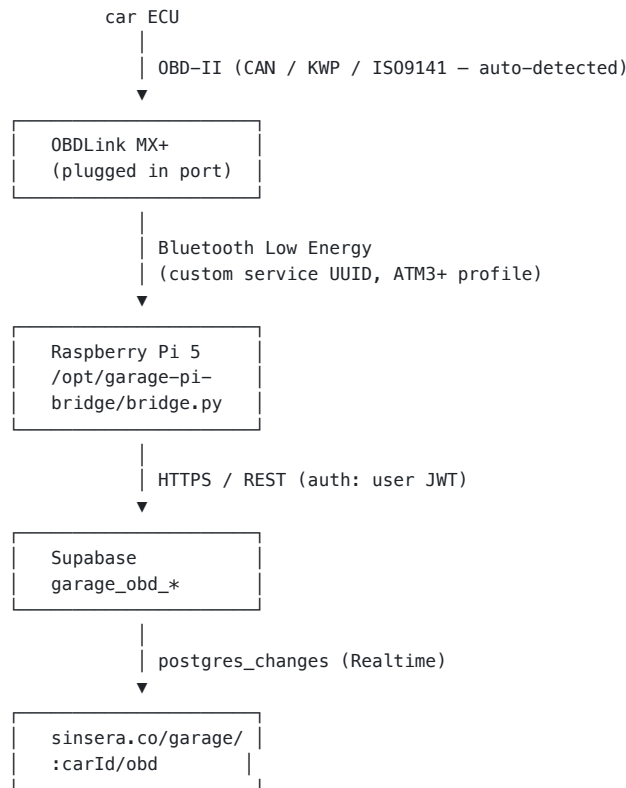
Open `sinsera.co/garage/<carId>/obd` in any browser — the gauges should fill in live.

## Troubleshooting

| Symptom                          | Likely cause               | Fix                                                                                   |
|----------------------------------|----------------------------|---------------------------------------------------------------------------------------|
| Bridge log: BLE device not found | Pi can't see the MX+       | Confirm MX+ has power (car ignition on). Re-pair via <code>bluetoothctl</code> .      |
| Bridge log: Supabase 401         | Auth failed                | Wrong email/password or the user can't write to that car.                             |
| Bridge log: ELM327 timeout       | OBD adapter not responding | Engine off, key not in run position, or MX+ firmware needs a reset (unplug + replug). |

| Symptom                                        | Likely cause                                             | Fix                                                                            |
|------------------------------------------------|----------------------------------------------------------|--------------------------------------------------------------------------------|
| Web app shows "Awaiting feed" but logs show OK | Realtime channel not subscribed for the right session_id | Check the GarageOBDPages selected session vs the one the bridge is writing to. |
| Pi reboots, bridge doesn't restart             | Service wasn't enabled                                   | <code>sudo systemctl enable garage-obd-bridge</code>                           |

## Architecture



## Files

- `bridge.py` — main service (asyncio, bleak, aiohttp)
- `obd_pids.py` — PID → decoder mapping (RPM, speed, coolant, etc.)
- `install.sh` — first-boot installer
- `garage-obd-bridge.service` — systemd unit
- `requirements.txt` — Python deps
- `.env.example` — required env vars

## License

MIT — do whatever you want with it.

## Releases

No releases published

## Packages

No packages published

---

## Contributors 1



**PetaStockdale** Peta Stockdale

---

## Languages

• Python 87.5% • Shell 12.5%